



Adam Ferreira

in ad-ferreira

QA Automation Architect

Je conçois ce qui rend une app mobile testable : framework, CI, données par API, test IDs dans le code, outillage pour les équipes.

🇫🇷 Paris, France · Freelance · sur site et télétravail

Mobile

Appium · Flutter · iOS · Android · BrowserStack

Frameworks

Java / TestNG / Cucumber · TypeScript / WebdriverIO · CodeceptJS / Playwright · Gherkin

Livraison & outillage

GitLab CI · Xray / Jira · Allure · mitmproxy · Claude Code · MCP Atlassian / Figma

EXPÉRIENCES PROFESSIONNELLES

QA Automation Engineer (mobile Flutter) — BETCLIC Mars 2026 - Aujourd'hui (Freelance)

Betclic reconstruit son app mobile iOS et Android en Flutter. J'y automatise les tests de la partie gamification : les missions et défis qui récompensent le joueur pour ses paris, les classements entre parieurs, les jeux de récompense qui en découlent et les boosts proposés au moment du pari. Le tout dans un repo de tests partagé entre le mobile et le web.

Construire l'automatisation en même temps que l'app

À mon arrivée, l'app était en construction, et le périmètre gamification avec elle. Le défi : automatiser au rythme des livraisons, sans ralentir les devs.

- **Des test IDs posés directement dans le code de l'app.** Sur mobile, un locator qui repère un écran par son texte casse à la première traduction ou au premier changement de maquette. J'ai été le premier QA auto de l'app à poser moi-même les semantics dans le code Flutter, selon des règles écrites avec les tech leads, plutôt que d'attendre les devs. Les mêmes locators servent sur iOS et Android, et ils survivent aux refontes d'écran.
- **Les animations interactives, jusque-là impossibles à tester.** Elles sont produites par l'équipe motion design dans un outil dédié, et les tests ne voyaient rien de ce qu'il y avait dedans. J'ai écrit un outil qui injecte les test IDs directement dans les fichiers d'animation, vérifié le résultat avec le runtime Flutter, puis produit une version que le designer ouvre dans son propre logiciel pour comprendre la manipulation et la refaire lui-même.
- **Des données de test créées par API, pas à la main.** Tester une mission de paris demande de vrais paris, placés puis réglés. Je les crée par API avant chaque scénario, à l'image des campagnes de production, sur un compte de test dédié. Le test ne passe par l'UI que pour ce qu'il vérifie vraiment : il est court, et il ne casse pas parce qu'un autre test a touché aux mêmes données.
- **Une nightly sur cinq marchés réglementés.** Chaque pays a ses règles, ses contenus et ses comptes. La nightly tourne sur chacun d'eux ; le matin, je triage les échecs en trois cas : bug produit, environnement en panne, test flaky. Seuls les premiers deviennent des tickets.
- **Remonter jusqu'au moteur mobile quand un test échoue sans raison.** Un bouton visible que le test ne pouvait pas taper, une tab bar qui disparaissait de l'arbre d'accessibilité iOS, la navbar système Android qui absorbait les taps : à chaque fois j'ai isolé la cause dans le comportement d'iOS, d'Android ou de Flutter, et corrigé dans le framework plutôt que de contourner dans les tests.
- **Une méthode partagée avec la squad.** Un test plan par feature, une suite de tests de release, et des skills Claude Code partagés qui génèrent les page objects à partir des maquettes Figma et des tickets Jira.

Stack : Flutter · CodeceptJS · Appium · Playwright · Cucumber/Gherkin · BrowserStack · Rive · Claude Code · MCP Atlassian / Figma

QA Automation Architect — ACCOR (CAC40) Nov. 2024 - Mars 2026 (Freelance)

Accor teste ses apps mobiles ALL (fidélité, réservation, séjour) avec un framework de tests automatisés de quatre ans, en dette technique. Arrivé pour exécuter et maintenir des tests, j'en suis devenu le référent technique.

Remettre un framework legacy à l'état de l'art, puis aller plus loin

- **Moderniser le framework sans arrêter les tests.** Migrations de Java et du client Appium enchaînées sans régression. J'en ai profité pour retirer le code mort et refaire les logs, lisibles scénario par scénario. Le framework a perdu la moitié de son poids.
- **Un device différent pour chaque scénario.** Les bugs propres à un modèle, à une tablette ou à un écran en paysage ne se voient pas quand tous les tests tournent sur le même téléphone. J'ai fait distribuer chaque scénario sur un device différent de la device farm BrowserStack : les échecs sont devenus reproductibles, et avec un cache des apps et des devices, les sessions démarrent plus vite.
- **Un pipeline CI qui dit enfin si la release est bonne.** Avant, il lançait les tests mais restait vert quoi qu'il arrive, et il reposait sur un template partagé avec d'autres projets, qu'on ne pouvait pas adapter. Je l'ai réécrit pour l'équipe : il passe au rouge dès que le taux de réussite descend sous un seuil, publie les résultats dans Jira et Xray, et prévient chaque équipe sur Slack de ses propres échecs.
- **Mettre à portée des tests ce qui leur échappait.** Les features derrière un feature flag, les builds store, les distributions TestFlight et Firebase, et les webviews derrière le WAF de l'entreprise.
- **Des bugs de l'app diagnostiqués pour les devs.** Le login iOS faisait échouer un run sur cinq. J'ai comparé des sessions qui passaient et des sessions qui échouaient sur le même build, isolé la cause, et travaillé le fix en binôme avec le tech lead iOS. Même approche pour les sessions expirées et les écrans de consentement absents.

- **Un second framework, en TypeScript, conçu de zéro.** Il n'existait pas de framework TypeScript pour le mobile chez Accor. J'en ai construit un, multi-plateforme et multi-locale, et j'ai documenté la comparaison avec la stack Java.

Stack : Java 25 · Maven · Appium · TestNG · Cucumber · BrowserStack · GitLab CI · Xray/Jira · TypeScript · WebDriverIO 9 · Allure

Frontend Software Developer — Hexaly (Bouygues) Mars 2024 - Mai 2024

Développement de fonctionnalités sur Hexaly Studio, un SaaS low-code React pour résoudre des problèmes d'optimisation complexes (planification de livraisons, gestion d'entrepôts logistiques).

Stack : React · TypeScript · JavaScript

Développeur Full Stack — JMD Technologies (Freelance) Avr. 2023 - Déc. 2023

Maintenance Industrie, spécialiste du nettoyage industriel, a commandé un SaaS de gestion de chantiers ainsi qu'une application mobile de badgeuse pour ses agents de nettoyage.

Stack : Angular · Express.js · Sequelize · MySQL · Flutter · Riverpod

FORMATION

ESIEE Paris — Ingénieur IMAC 2020 - 2023

Spécialisation web et mobile — Université Gustave Eiffel.

LANGUES

Français — Langue maternelle · **Anglais** — Professionnel avancé, niveau C1 (TOEIC 975)

CENTRES D'INTÉRÊT

Escalade — Haut niveau : 8a en voie, 7C en bloc.